Implementación de una aplicación web con Spring e Hibernate

Desarrollo Entorno Servidor

Estructura

En bloque de diapositivas vamos a explicar la arquitectura en la aplicación de **springboot_hibernate** disponible en [GitHub](), utilizando código.

La arquitectura empleará:

- Capa de Servicios
- Capa de acceso a datos con DAO
- Uso de Hibernate

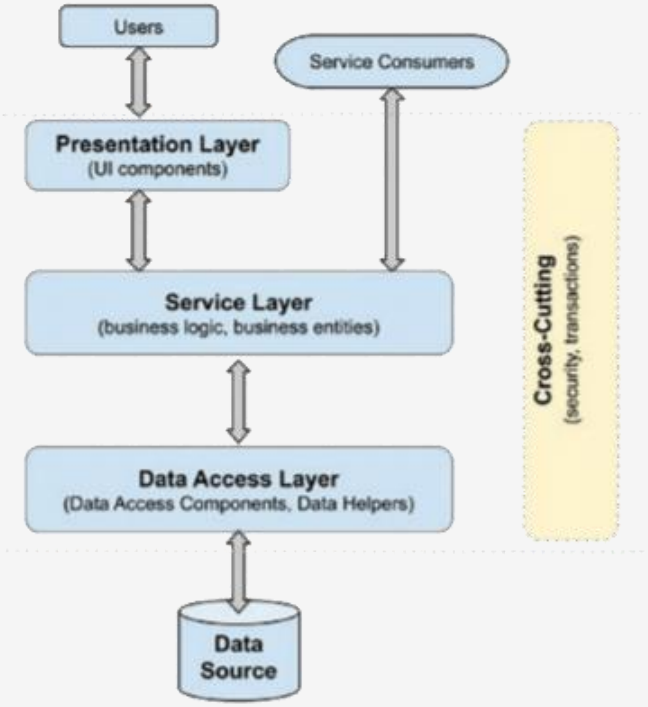


Figure 5-1. *Typical software application architecture*

Aplicación

La aplicación intenta representar la aplicación web de una entidad educativa en la que se puede consultar los módulos que se imparten y los profesores asociados a ellos.

Los profesores tienen asociados de uno a varios emails. Estos son únicos y no puede existir otro profesor con el que comparta email.

Los profesores pueden impartir uno o varios módulos, y estos módulos pueden ser impartidos por uno o varios profesores.

Los profesores tienen una única foto de perfil.

Aplicación

La aplicación intenta representar la aplicación web de una entidad educativa en la que se puede consultar los **módulos** que se imparten y los **profesores** asociados a ellos.

Los **profesores** tienen asociados de uno a varios **emails**. Estos son únicos y no puede existir otro **profesor** con el que comparta **email**. **(1:N)**

Los **profesores** pueden impartir uno o varios **módulos**, y estos **módulos** pueden ser impartidos por uno o varios **profesores**. **(N:M)**

Los **profesores** tienen una única **imagen** de perfil. **(1:1)**

Pom.xml

Antes de nada vamos a añadir las librerías que necesitamos:

```
<dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
</dependency>
<dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-
      thymeleaf</artifactId>
</dependency>
<dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.21</version>
</dependency>
<dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Application.properties

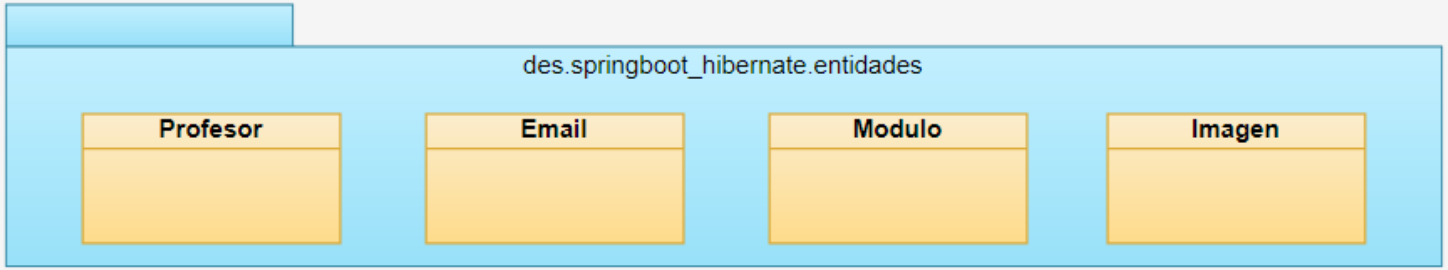
Configuramos el DataSource

```
server.port=9090
spring.thymeleaf.cache=false

# MySQL
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost/springboot_hibernate?allowPublicKeyRetr
ieval=true&createDatabaseIfNotExist=true&useUnicode=true&useJDBCCompliantTimezoneShi
ft=true&useLegacyDatetimeCode=false&serverTimezone=UTC&autoReconnect=true&useSSL=fal
se
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.initialization-mode=always

#Turn Statistics on and log SQL stmts

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

#If want to see very extensive logging
spring.jpa.properties.hibernate.generate_statistics=true
logging.level.org.hibernate.type=trace
logging.level.org.hibernate.stat=debug

#Schema will be created using schema.sql and data.sql files

spring.jpa.hibernate.ddl-auto=create

#Increasing image file sizeupload
spring.servlet.multipart.max-file-size=2048KB
spring.servlet.multipart.max-request-size=2048KB
```

- Añadimos distintas instrucciones para mostrar las consultas que se ejecutan por detrás en la consola
- `spring.jpa.hibernate.ddl-auto=create`
  - Ejecuta el script de creación de tables llamado **schema.sql**
  - Ejecuta el script de inserción de datos en tablas llamado **import.sql**
  - **https://stackoverflow.com/questions/42135114/how-does-spring-jpa-hibernate-ddl-auto-property-exactly-work-in-spring**

Entidades

Creamos una clase para definir cada entidad. Las entidades van a representar los objetos de dominio con los que va trabajar nuestra aplicación. Estos serán nuestros POJOs:



Las entidades tienen que cumplir las siguientes características:

- Tener un constructor público sin ningún tipo de argumentos.
- Método Getter y Setter para cada atributo que queramos.
- La anotación **@Entity** encima de la clase.
- Implementar el interfaz **Serializable**

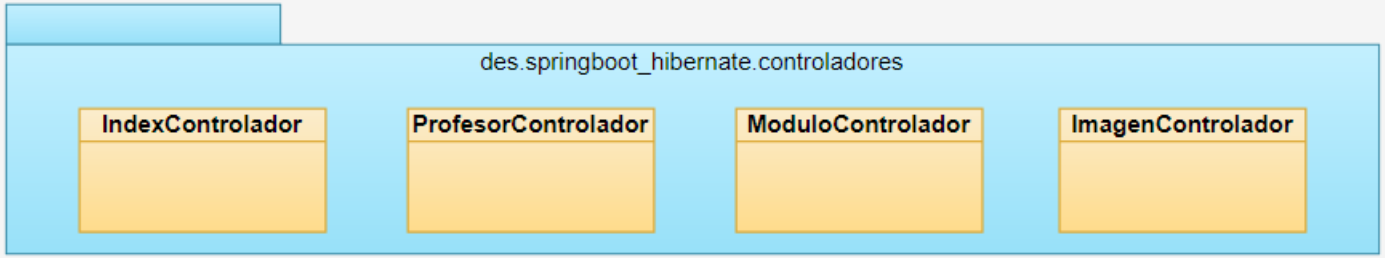
Entidades.- Profesor

```java
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

private static final long serialVersionUID = -8668594760203621162L;

@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@Column(name = "NICKNAME")
private String username;

@Column(name = "NOMBRE")
private String nombreProfesor;

@Column(name = "APELLIDOS")
private String apellidosProfesor;

@OneToOne(mappedBy="profesor" ,fetch = FetchType.LAZY,optional= true)
private Imagen imagen;

@OneToMany(fetch = FetchType.EAGER,mappedBy = "profesor", cascade = CascadeType.ALL,
orphanRemoval = true)
private Set<Email> emails = new HashSet<>();

@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(name = "PROFESOR_MODULO",
joinColumns = @JoinColumn(name = "ID_PROFESOR"),
inverseJoinColumns = @JoinColumn(name = "ID_MODULO"))
private Set<Modulo> modulos = new HashSet<>();
```

```java
public Set<Modulo> getModulos() {
return modulos;
}

public void setModulos(Set<Modulo> modulos) {
this.modulos = modulos;
}

public Set<Email> getEmails() {
return emails;
}

public void setEmails(Set<Email> emails) {
this.emails = emails;
}

public Long getIdProfesor() {
return idProfesor;
}

public String getNombreProfesor() {
return nombreProfesor;
}

public String getApellidosProfesor() {
return apellidosProfesor;
}

public void setIdProfesor(Long idProfesor) {
this.idProfesor = idProfesor;
}
```

Controladores

Diseñamos los controladores en función de las peticiones que vamos a recibir y nuestros modelo del dominio:



Como se puede ver a diferencia de la entidades tenemos un **IndexController** pero no tenemos un **EmailController.** El motivo es que la petición de carga inicial de la página no se relaciona con ningún otro controlador por ello tiene su propio controlador.

Los emails solo dependen del profesor, los emails tendrán las operaciones CRUD, sin embargo cada vez que accedamos al email será a través del perfil de un profesor por ello las peticiones que sean para los emails estas serán procesadas por el controlador del Profesor.

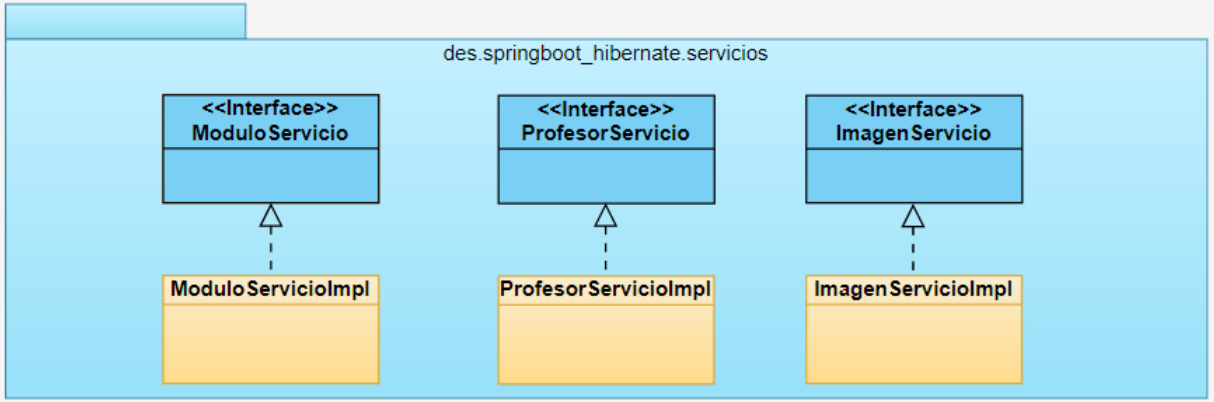
Controladores

```
@Controller
@RequestMapping(value = "/profesor")
public class ProfesorControlador {

@Autowired
ProfesorServicio profesorService;

@Autowired
ModuloServicio moduloServicio;

@RequestMapping(method = RequestMethod.GET, value = "/lista")
public ModelAndView listarPorfesores() {

	ModelAndView mav = new ModelAndView();

	List<Profesor> lProfesor = profesorService.listarProfesores();

	mav.addObject("profesores", lProfesor);
	mav.setViewName("profesor/lista");
return mav;
}
@GetMapping("/actualizar/{id}")
public ModelAndView mostrarActualizarPerfilProfesor(
@PathVariable("id") long idProfesor, HttpServletRequest request) {

	Profesor profesor = profesorService.obtenerProfesor(idProfesor);
	List<Modulo> lModulos = moduloServicio.listarModulos();
	ModelAndView mav = new ModelAndView();
	mav.addObject("profesor", profesor);
	mav.addObject("modulos", lModulos);
	mav.setViewName("profesor/actualizar");
return mav;
}
```

- Anotamos las clase con **@Controller**.
- Anotamos la clase con @RequestMapping, con esta anotación la clase responderá solo a las peticiones de http://ww.midominio**/profesor**
- El controlador instancia los servicios de los que requiere solicitar información y los anota con **@Autowired**.

Servicios.

Los servicios los diseñamos en función de la lógica de negocio de nuestra aplicación. De la misma manera que las peticiones sobre los emails eran procesadas por el controlador de profesor su lógica de negocio se llevará a cabo en el servicio de **ProfesorServicio**.



Servicios

Declaramos la interfaz con la operaciones que vamos a utilizar.

Como podemos fijarnos todas pueden ser encuadradas dentro del las operaciones CRUD;

```
public interface ProfesorServicio {

public Profesor crearPorfesor(Profesor porfesor);

public void eliminarPorfesor (long idProfesor);

public Profesor obtenerProfesor(long idProfesor);

public Profesor modificarProfesor(Profesor profesor);

public List<Profesor> listarProfesores();

public List<Profesor> listarPorfesoresQueNoImparten(Long idModulo);

public Profesor anadirEmail(long idProfesor, Email email);

public void eliminarEmail(long idProfesor, Email email);
}
```

Servicios

Implementamos el servicio.

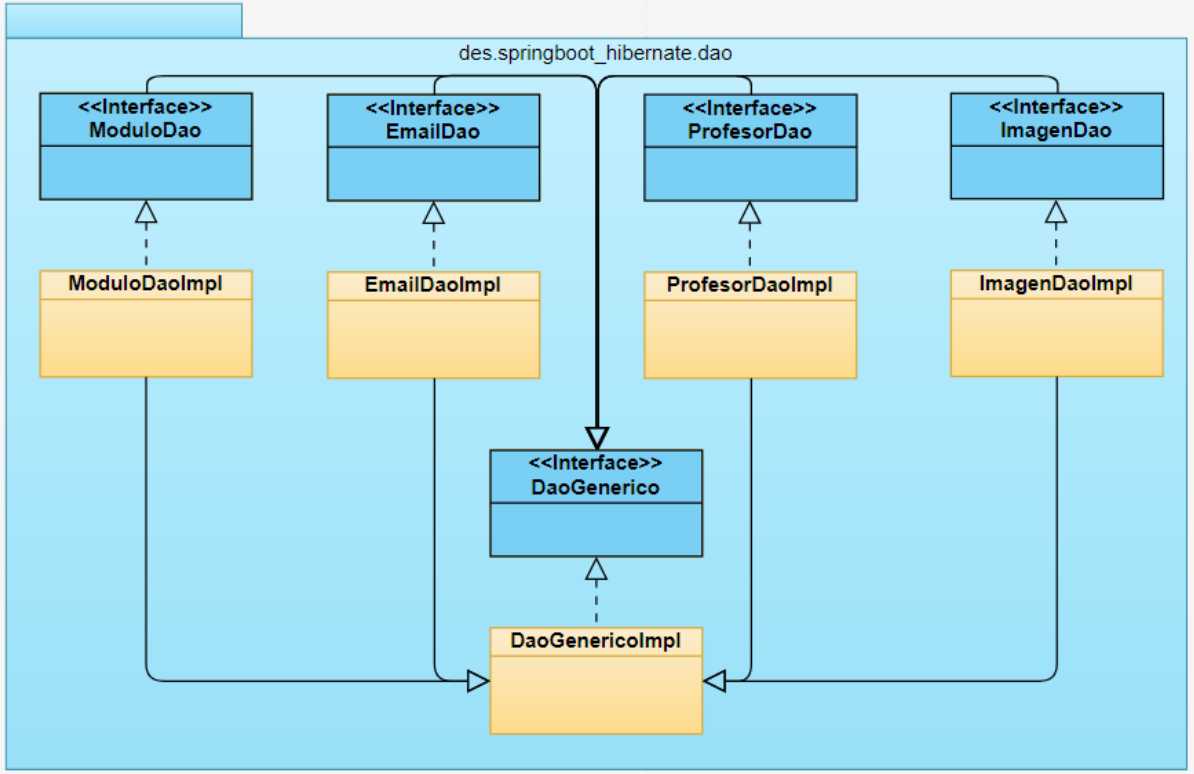
```
@Transactional
@Service
public class ProfesorServiceImpl implements ProfesorServicio {

@Autowired
private ProfesorDao profesorDao;

@Autowired
private EmailDao emailDao;

@Autowired
private ModuloDao moduloDao;

@Override
public List<Profesor> listarProfesores() {
	return profesorDao.listarProfesores();
}

@Override
public Profesor obtenerProfesor(long idProfesor) {
	return profesorDao.buscar(idProfesor);
}

@Override
public void eliminarEmail(long idProfesor, Email email) {
	Profesor p = profesorDao.buscar(idProfesor);
	p.eliminarEmails(email);
	emailDao.actualizar(email);
}
```

- Anotamos la clase con **@Transactional** y con **@Service**.
- Implementamos el interfaz donde hemos declarado las operaciones del Servicio.
- En el Servicio desarrollamos la lógica de negocio de una funcionalidad es por ello que puede que necesitemos acceder a DAOs de otras entidades.
- Por cada DAO que vayamos a necesitar lo instanciamos usando la anotación **@Autowired**.

DAO

Por cada entidad declaramos un DAO. Ya que estos son los únicos que van a manejar el acceso a datos de cada entidad.

Además añadimos un DAO genérico



¿DAO Genérico?

Como hemos mencionado todas las entidades deben tener los métodos CRUD.

Estos métodos se declaran de la misma manera ya que todo se realiza a través del **EntityManger**. El **EntityManger** los declara de manera genérica

<T> T	find(Class<T> entityClass, Object primaryKey) Find by primary key.
<T> T	find(Class<T> entityClass, Object primaryKey, LockModeType lockMode) Find by primary key and lock.
<T> T	find(Class<T> entityClass, Object primaryKey, LockModeType lockMode, Map<String,Object> properties) Find by primary key and lock, using the specified properties.
<T> T	merge(T entity) Merge the state of the given entity into the current persistence context.
void	persist(Object entity) Make an instance managed and persistent.
void	remove(Object entity) Remove the entity instance.
void	setFlushMode(FlushModeType flushMode) Set the flush mode that applies to all objects contained in the persistence context.
void	setProperty(String propertyName, Object value) Set an entity manager property or hint.
<T> T	unwrap(Class<T> cls) Return an object of the specified type to allow access to the provider-specific API.

https://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html

¿DAO Genérico?

Si el **EntityManager** define estos métodos como genéricos ¿porque vamos redefinir estos métodos para cada DAO?.

La solución un DAO genérico, que declare las funciones básicas (CRUD) para todas las entidades.

Como hacerlo:

- Declarando los atributos como object.
- Usando WildCards.

```
public interface DaoGenerico<T> {

    long contarTodos(Map<String, Object> params);

    T crear(T t);

    void borrar(Object id);

    T buscar(Object id);

    T actualizar(T t);
}
```

¿DAO Genérico?

Misma metodología declaramos un interfaz para las operaciones y una clase abstracta con las implementaciones.

```java
public abstract class DaoGenericoImpl<T> implements DaoGenerico<T> {

@PersistenceContext
protected EntityManager em;

private Class<T> type;

public DaoGenericoImpl() {
	Type t = getClass().getGenericSuperclass();
	ParameterizedType pt = (ParameterizedType) t;
	type = (Class) pt.getActualTypeArguments()[0];
}

@Override
public long contarTodos(final Map<String, Object> params) {
	final StringBuffer queryString = new StringBuffer("SELECT count(o) from ");
	queryString.append(type.getSimpleName()).append(" o ");
	queryString.append(this.getQueryClauses(params, null));
	final Query query = this.em.createQuery(queryString.toString());
	return (Long) query.getSingleResult();
}

@Override
public T crear(final T t) {
	this.em.persist(t);
	return t;
}

@Override
public T buscar(final Object id) {
return (T) this.em.find(type, id);
}

@Override
public T actualizar(final T t) {
return this.em.merge(t);
}

@Override
public void borrar(final Object id) {
	this.em.remove(this.em.getReference(type, id));
}
```

¿DAO Genérico?

Misma metodología declaramos un interfaz para las operaciones y una clase abstracta con las implementaciones.

```java
public abstract class DaoGenericoImpl<T> implements DaoGenerico<T> {

@PersistenceContext
protected EntityManager em;

private Class<T> type;

public DaoGenericoImpl() {
	Type t = getClass().getGenericSuperclass();
	ParameterizedType pt = (ParameterizedType) t;
	type = (Class) pt.getActualTypeArguments()[0];
}

@Override
public long contarTodos(final Map<String, Object> params) {
	final StringBuffer queryString = new StringBuffer("SELECT count(o) from ");
	queryString.append(type.getSimpleName()).append(" o ");
	queryString.append(this.getQueryClauses(params, null));
	final Query query = this.em.createQuery(queryString.toString());
	return (Long) query.getSingleResult();
}

@Override
public T crear(final T t) {
	this.em.persist(t);
	return t;
}
```

```java
@Override
public T buscar(final Object id) {
return (T) this.em.find(type, id);
}

@Override
public T actualizar(final T t) {
return this.em.merge(t);
}

@Override
public void borrar(final Object id) {
	this.em.remove(this.em.getReference(type, id));
}
```

Implementación del DAO Genérico

Como la clase **DAOGenérico** es abstracta las Implementaciones de los DAO de Profesor, Email, Módulo e Imagen no tendrán que implementar dichos métodos ya que son heredados de la clase padre.

```
public interface ProfesorDao  extends DaoGenerico<Profesor>{

}
```

```
public interface DaoGenerico<T> {

	long contarTodos(Map<String, Object> params);

	T crear(T t);

	void borrar(Object id);

	T buscar(Object id);

	T actualizar(T t);
}
```

```
public abstract class DaoGenericoImpl<T> implements DaoGenerico<T> {
}
```

```
@Repository
@Component("ProfesorDao")
public class ProfesorDaoImpl extends DaoGenericoImpl<Profesor> implements ProfesorDao {

}
```

DaoProfesor

Si observamos las operaciones de **PorfesorDao** No existen operaciones básicas como crear, borrar, obtener por id…

Estas operaciones son heredadas del **DaoGenerico**

```
public interface ProfesorDao  extends DaoGenerico<Profesor>{

       public Profesor buscarPorEmail (String email);

       public Profesor buscarPorNombre(String username);

       public Profesor anadirEmail(long idProfesor,Email email);

       public List<Profesor> listarProfesores();

       public List<Profesor>listarPorfesoresQueNoImparten(List<Profesor>
       lprofesores);

       public Modulo desmatricularProfesor(Modulo modulo, Profesor profesor);
}
```

DaoProfesorImpl

```java
@Repository
@Component("ProfesorDao")
public class ProfesorDaoImpl extends DaoGenericoImpl<Profesor> implements
ProfesorDao {

@Override
public Profesor buscarPorEmail(String email) {
Query query =
this.em.createQuery("select u FROM Profesor u where u.email= :email");
query.setParameter("email", email);
Profesor profesor = (Profesor) query.getSingleResult();

return null;
}

@Override
public List<Profesor> listarProfesores() {
Query query = this.em.createQuery("FROM Profesor");
List<Profesor> lProfesor = query.getResultList();

return null;
}
```

Para la implementación del DAO:

- Anotamos la clase con **@Repository** para indicar que se encarga de la persistencia.
- Anotamos con **@Component** para indicar que es una posible implementación de **PorfesorDao**
- Como se puede observar en el DAO No se realiza la llamada de otros DAO.
  - Principio de responsabilidad única cada DAO solo maneja la persistencia de su clase asociada y no maneja la de las otras entidades.
  - Esto sería responsabilidad del servicio.